Workplace Ninjas Australia

SHARING KNOWLEDGE. TOGETHER.

WORKPLACE NINJA



Workplace Ninjas Australia

What's up with WinGet?

Tom Plant | Devicie

Workplace Ninjas Melbourne 2026

Thank you, Melbourne sponsors!

glueck kanja

DEVICIE

ENDPOINTFOCUS

Tom Plant

Head of Devicie Labs

Software engineering & security

~6 years in security

Industry, government, education

Open source

Vulnerability research



Homelab

Kubernetes

Competitive hacking

Capture-the-flag (CTF)

Connect

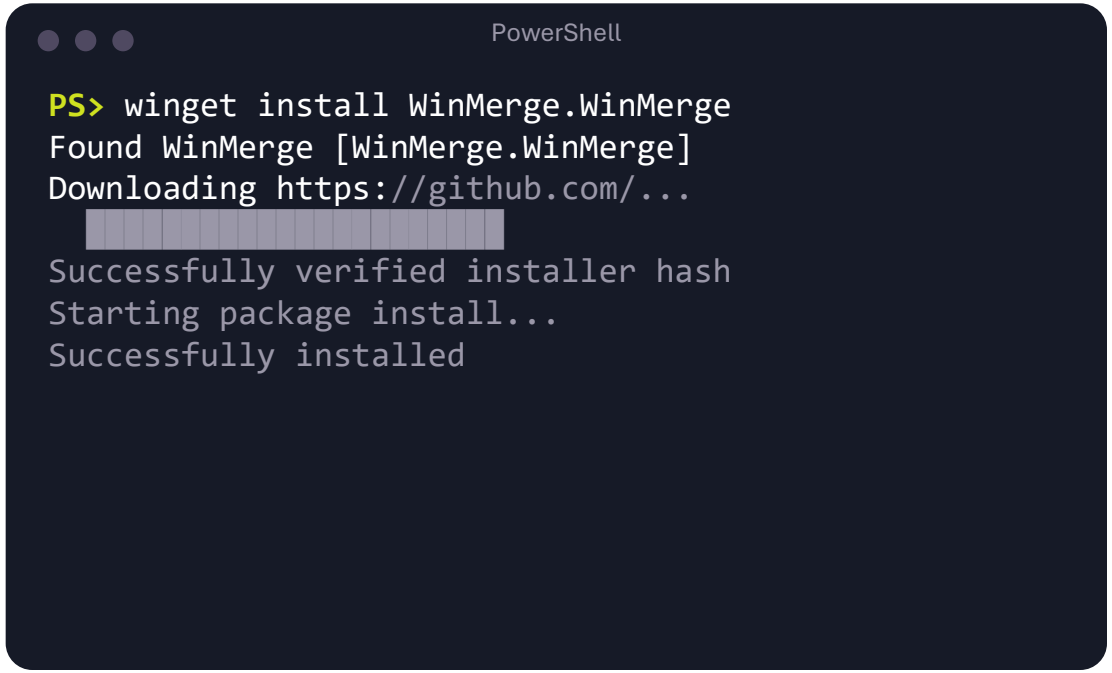
Hands up!

Who's heard of WinGet?

~~winget~~ Windows Package Manager

An open-source package manager that finds, installs, upgrades and removes apps. It ships in App Installer on Windows 10 1809+ and Windows 11.

- CLI: winget.exe
- COM API + PowerShell module
- WinGet Configuration (DSC)
- MCP server for AI agents



How WinGet can help you

Store apps, no packaging

Intune

Add a "Microsoft Store app (new)" and the IME installs and updates it with the WinGet engine, Win32 Store apps included.

Patch every client tenant

MSPs

RMMs like NinjaOne use WinGet to install and update third-party apps across all your customers.

Fill the patching gap

Free

Deploy Winget-AutoUpdate as an Intune app: it updates your allow-listed apps daily as SYSTEM.

Win32 apps, pre-built

Intune

Partners like Robopack turn WinGet packages into tested Intune Win32 apps, with update rings.

Before WinGet

Mar 2011

Chocolatey

Rob Reynolds. NuGet packages + PowerShell install scripts. Repackages or wraps installers.

May 2013

Scoop

Luke Sampson. JSON manifests, portable user-scope apps, no admin rights.

2014

AppGet

Keivan Beigi. YAML manifests on GitHub, vendor installers. WinGet borrowed the design.

Also studied by the WinGet team: Ninite · Npackd · OneGet / PackageManagement

WinGet milestones

2015

OneGet

PackageManagement ships in Windows 10

2016

App Installer

the app WinGet ships in today

Jul 2019

Repos created

winget-cli + winget-pkgs, private

May 2020

Preview

announced at Build

May 2021

v1.0

1,400 packages

Late 2022

Intune

Store app (new) uses WinGet

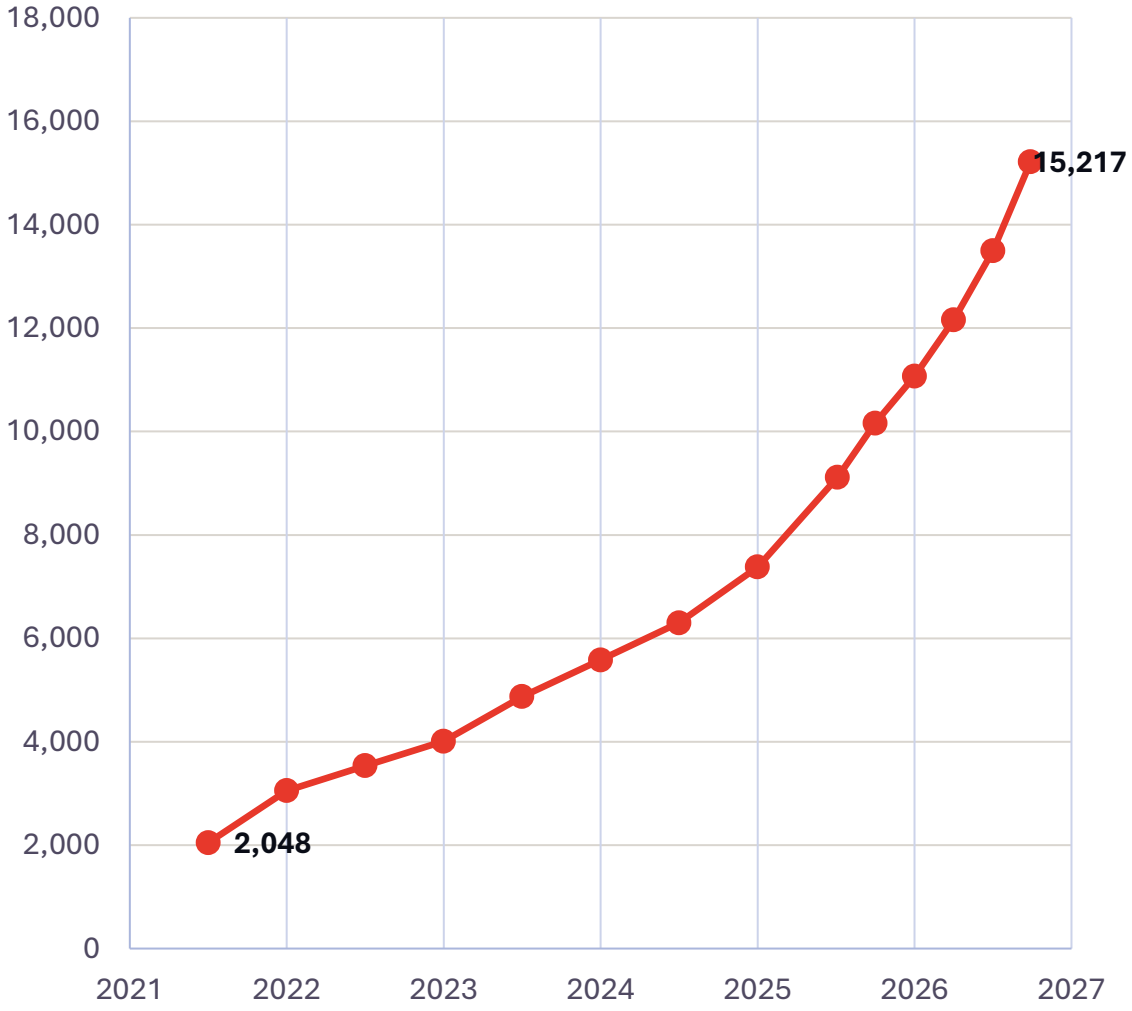
2023

Configure

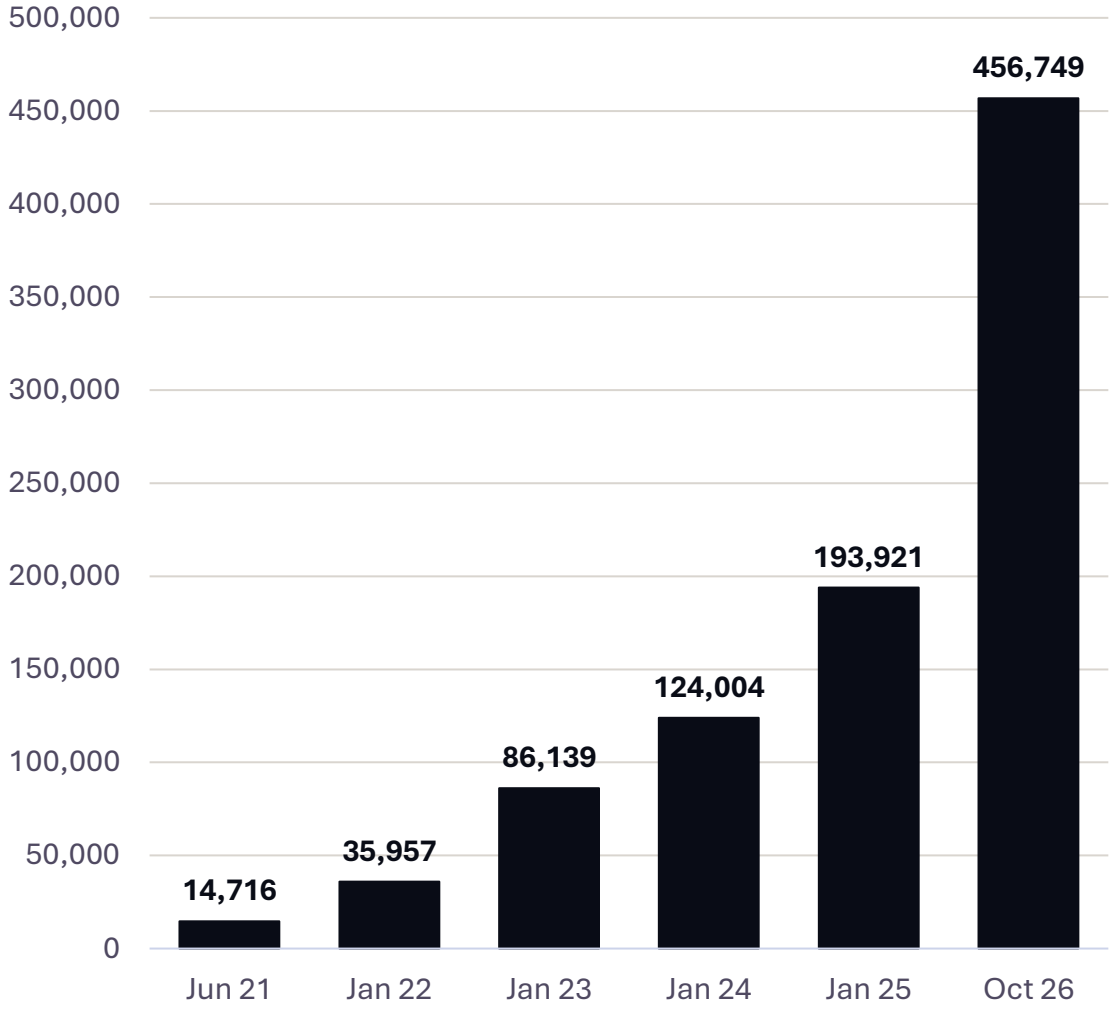
DSC YAML for machine setup

Growth

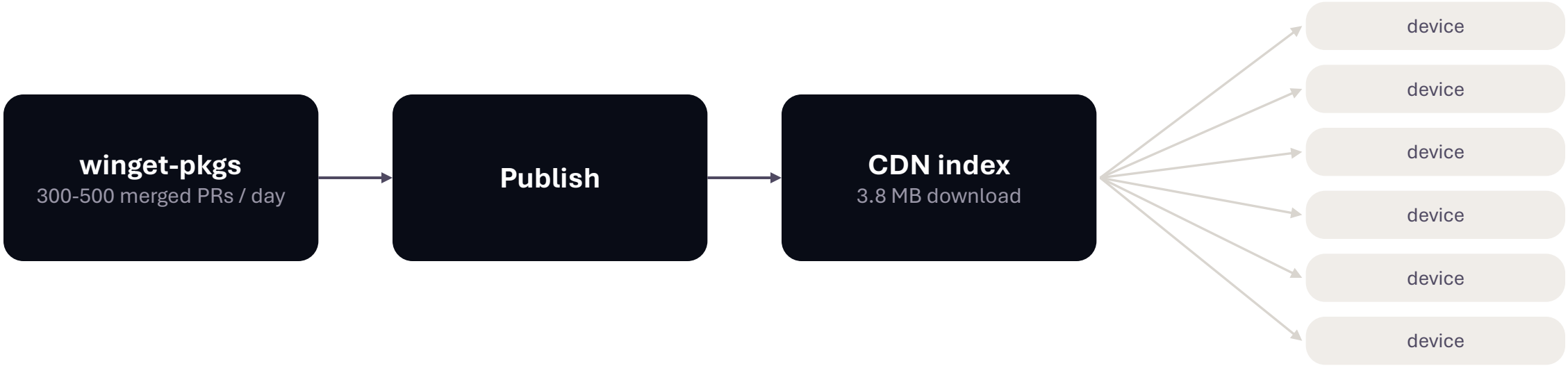
Packages in microsoft/winget-pkgs



Pull requests to winget-pkgs (cumulative)



Success has a cost



The index

Every refresh downloads the whole index. Delta indexes are in progress.

The maintainers

New packages wait for a human: over a month, with AI agents flooding the queue.

For you

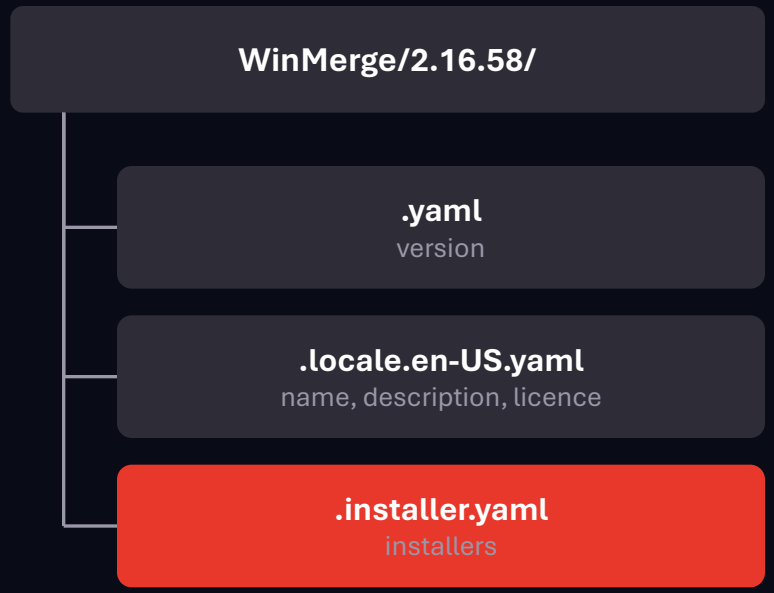
New versions land whenever a PR merges. There is no change window.

Under the hood

Manifests, installs and detection

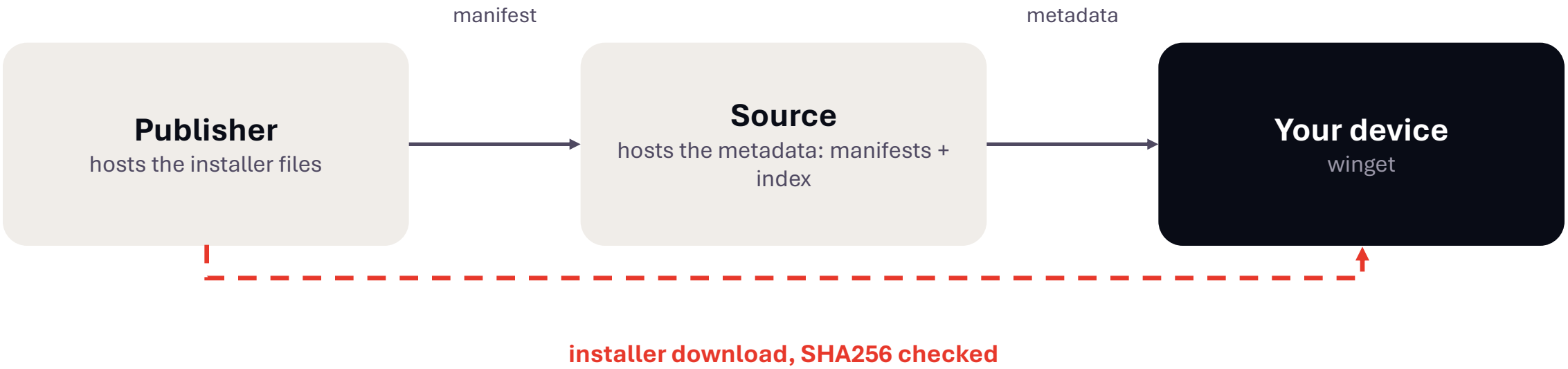
Example: WinMerge

```yaml
# Created using wingetcreate 1.12.8.0
PackageIdentifier: WinMerge.WinMerge
PackageVersion: 2.16.58
InstallerType: inno
ProductCode: WinMerge_is1
Installers:
- Architecture: x86
  Scope: machine
  InstallerUrl: https://github.com/.../WinMerge-2.16.58-Setup.exe
  InstallerSha256: 41E7CA58694F...A7F1D5F304
  ElevationRequirement: elevatesSelf
- Architecture: x64
  Scope: user
  InstallerUrl: https://github.com/.../WinMerge-...-PerUser-Setup.exe
  InstallerSha256: 7F899B7853AF...AF74A73EC
- Architecture: x64
  Scope: machine
  InstallerUrl: https://github.com/.../WinMerge-2.16.58-x64-Setup.exe
  InstallerSha256: AF00357FDBBF...CCE28F2AB3
  ElevationRequirement: elevatesSelf
- Architecture: arm64
  Scope: machine
  # ...
ManifestType: installer
ManifestVersion: 1.12.0
```



Generate these with Komac, wingetcreate or YamlCreate. Don't hand-write them.

Who hosts what



Microsoft only hosts metadata, installers come from the publisher

Just look at the source!

```
// Workflows/InstallFlow.cpp (simplified)
void InstallSinglePackage(Execution::Context& context)
{
    context <<
        ShowPromptsForSinglePackage(true) <<
        InstallDependencies <<
        DownloadInstaller <<      // download, hash, AV scan
        InstallPackageInstaller;
}

void InstallPackageInstaller(Execution::Context& context)
{
    context <<
        SnapshotARPEntries <<     // before
        ExecuteInstaller <<       // msiexec, exe, msix...
        ReportARPChanges <<       // after: what changed?
        RecordInstall;            // save to installed.db
}
```

Open source (MIT): every line of the client is on GitHub, so you can read exactly what it does. No decompiling .NET assemblies like you would for the Intune agents.

winget install, step by step

Find → **Choose** → **Download** → **Install**

open sources, pick a package + version

filter and rank the installers

SHA256 check, Mark-of-the-Web + AV scan

run it, then snapshot + correlate ARP

Every installer in the manifest

Filter: MinOSVersion, market

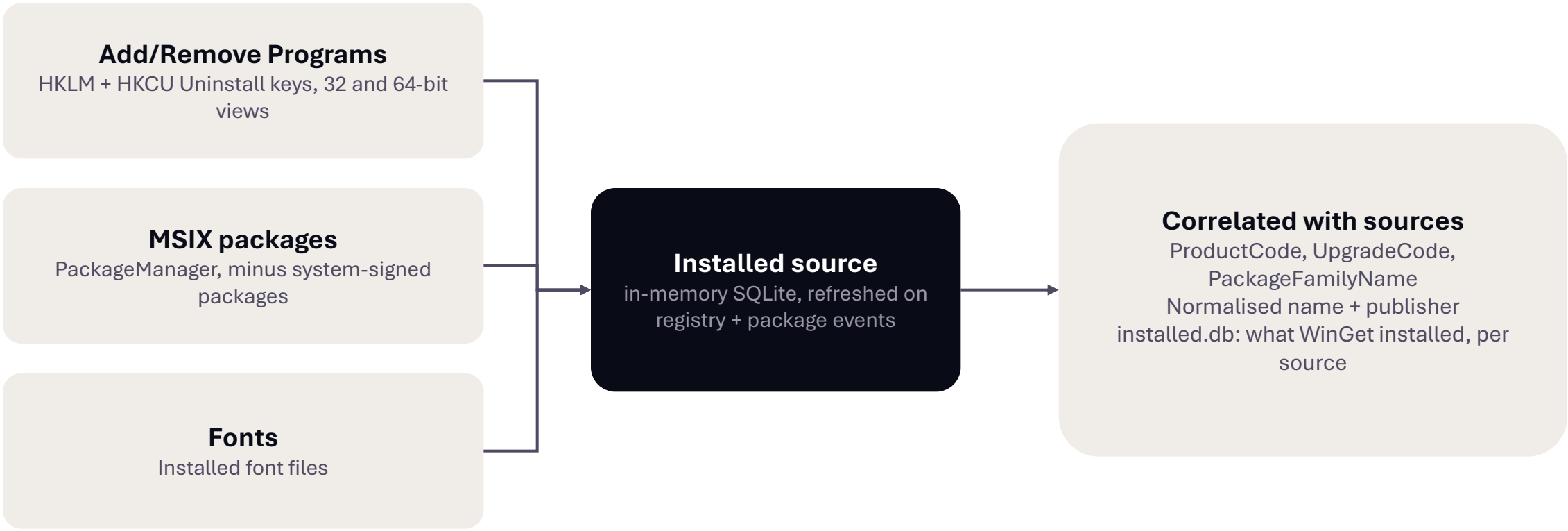
Filter: installed scope + type (on upgrade)

Rank: locale > scope > arch > type

One installer

Type	Silent switches / handling
MSI / WiX / Burn	`/quiet /norestart`
Inno	`/SP- /VERYSILENT /SUPPRESSMSGBOXES /NORESTART`
Nullsoft	`/S`
EXE	`(none - manifest must supply)`
MSIX	Windows deployment API, can stream from the URL
Portable	copied to WinGet\Packages, alias in WinGet\Links (on PATH)
Zip	extracted, then the nested installer runs

What's installed?



Invisible to WinGet: SystemComponent = 1, no DisplayName, or no version.

Which ARP entry is mine?

Before / after snapshot, then:

1. Exactly one entry changed AND matches the manifest
2. Exactly one entry matches (unchanged)
3. Exactly one entry changed at all
4. Fuzzy match on name + publisher

Heuristic (step 4)

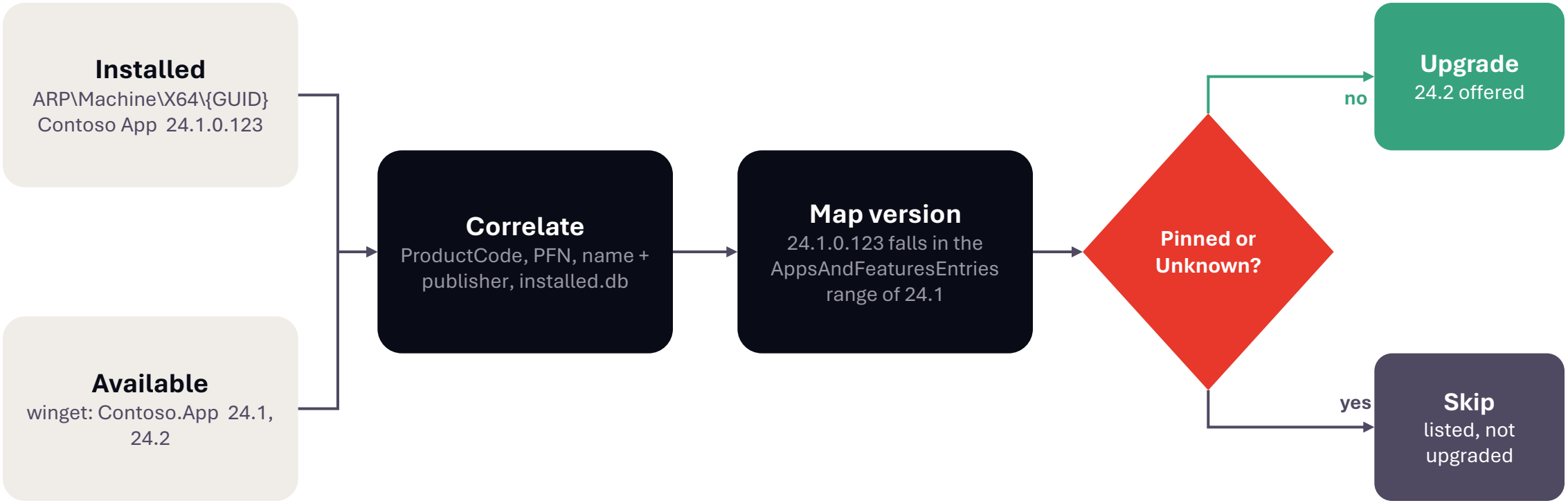
```
// ARPCorrelation.cpp / ARPCorrelationAlgorithms.h
score = 0.8 * similarity(name, publisher)
      + 0.2 * (entry is new ? 1 : 0)

// name weighted 2/3, publisher 1/3,
// by word edit distance

accept if  best >= 0.5
      and  best - runnerUp >= 0.05
```

Misses are fixed in the manifest: **AppsAndFeaturesEntries**.

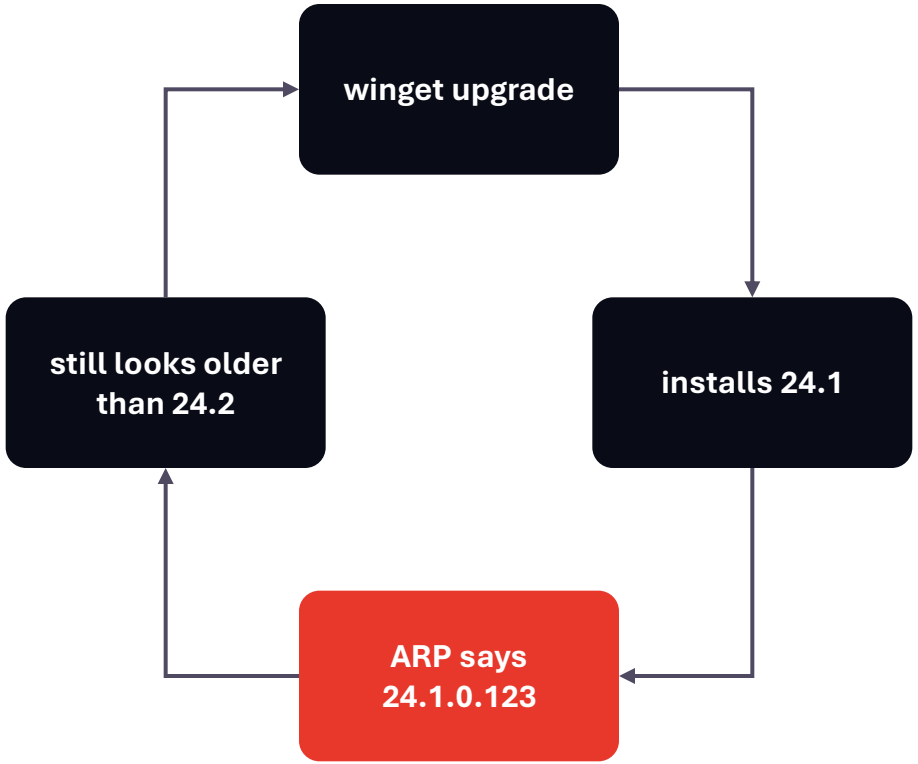
How winget upgrade decides



Version strings are hard

Comparison	Result	Why
`v2.0 vs 2.0`	equal	leading non-digits trimmed
`1.0 vs 1.0.0`	equal	trailing .0 parts trimmed
`1.0 vs 1.0-beta`	1.0 is newer	no suffix > suffix
`Unknown vs anything`	Unknown is older	but upgrade skips it

The upgrade loop



Fix: AppsAndFeaturesEntries.DisplayVersion in the manifest, so 24.1.0.123 maps to 24.1.

Demo

Install WinMerge (user scope)



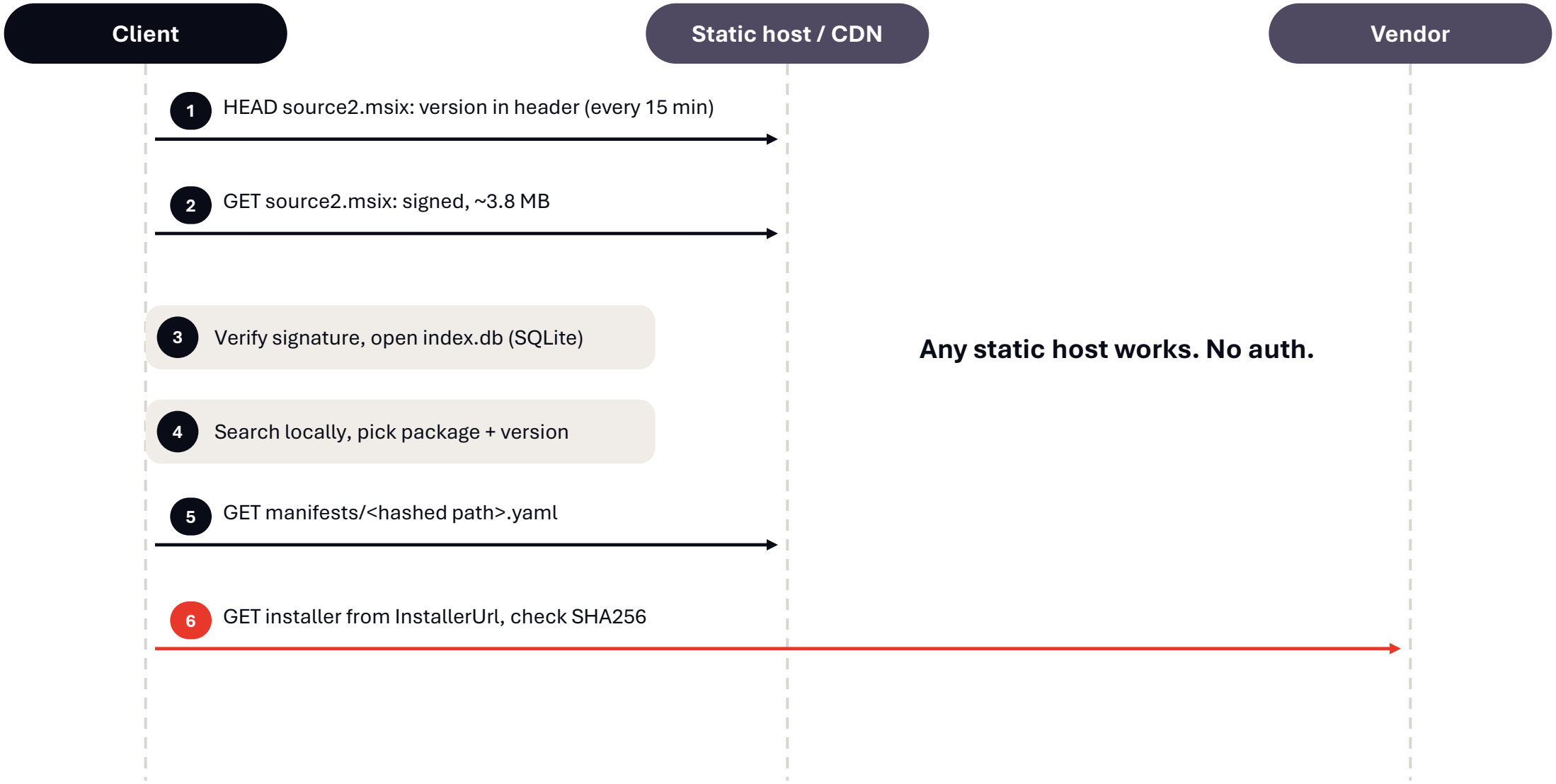
Sources

Where the catalogue lives, and how the client reads it

Out of the box

Name	Type	Endpoint	Notes
winget	PreIndexed	`cdn.winget.microsoft.com/cache`	the community repo
msstore	REST	`storeedgefd.dsx.mp.microsoft.com/v9.0`	certificate-pinned
winget-font	PreIndexed	`cdn.winget.microsoft.com/fonts`	explicit: only used with -s

Pre-indexed source, on the wire

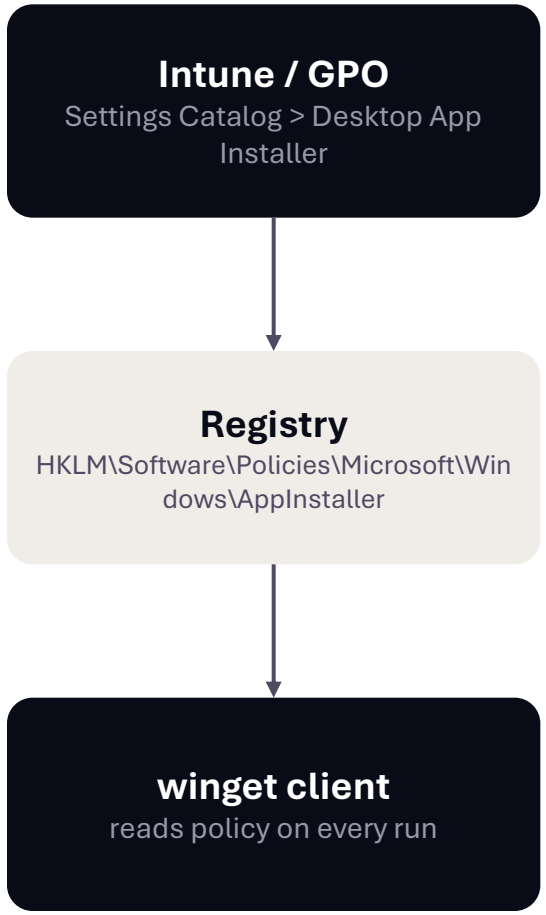


REST source, on the wire



Real per-user auth, but Entra ID sign-in fails as SYSTEM.

Controlling sources with policy



EnableDefaultSource / EnableMicrosoftStoreSource

switch the community / Store sources off

EnableAdditionalSources

push your own sources to every device

EnableAllowedSources

allow-list what users may add

SourceAutoUpdateInterval

change the 15-minute refresh

EnableHashOverride / EnableLocalManifestFiles

keep these risky flags off

EnableWindowsPackageManagerMcpServer

allow or block the MCP server

The community source

microsoft/winget-pkgs: how a version gets to your devices

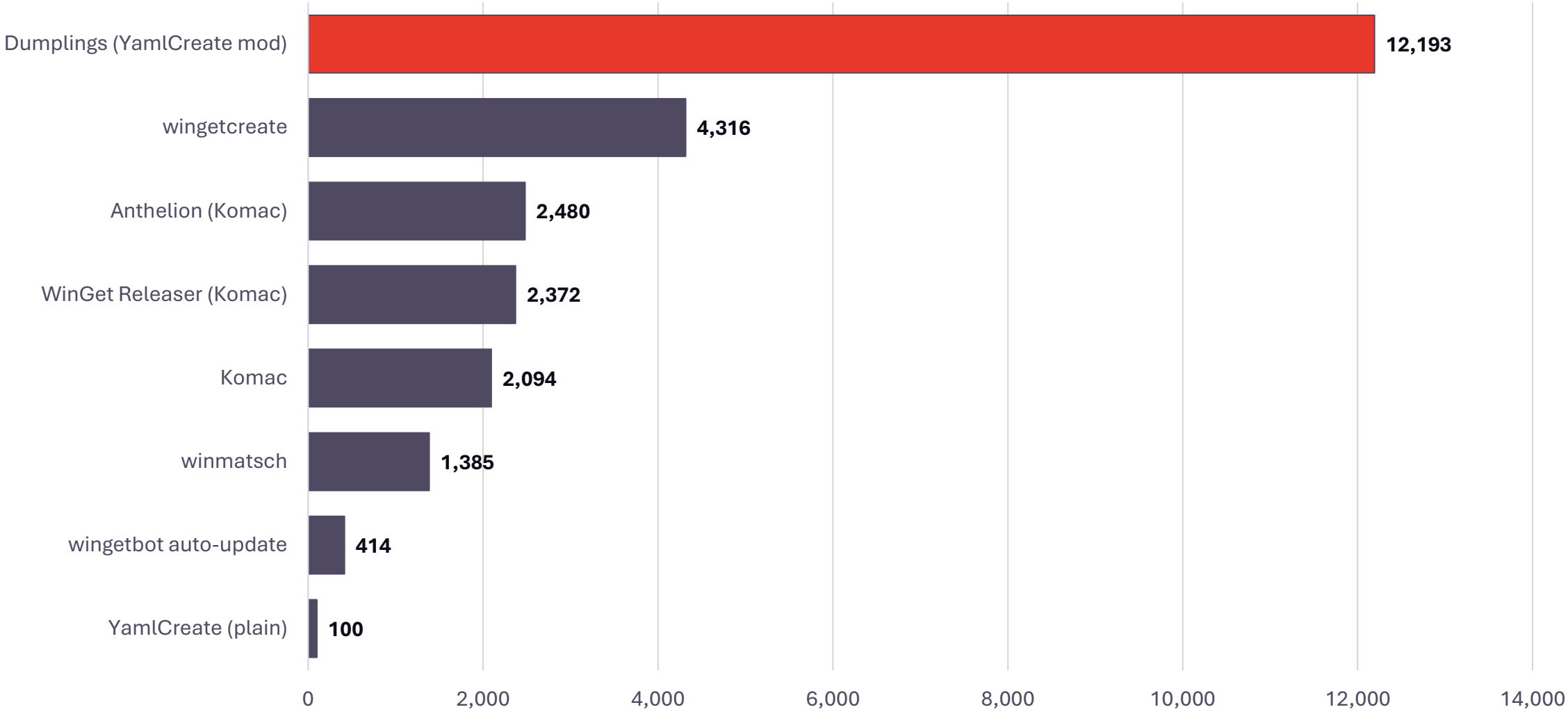
Demo

Submit a new version with Komac

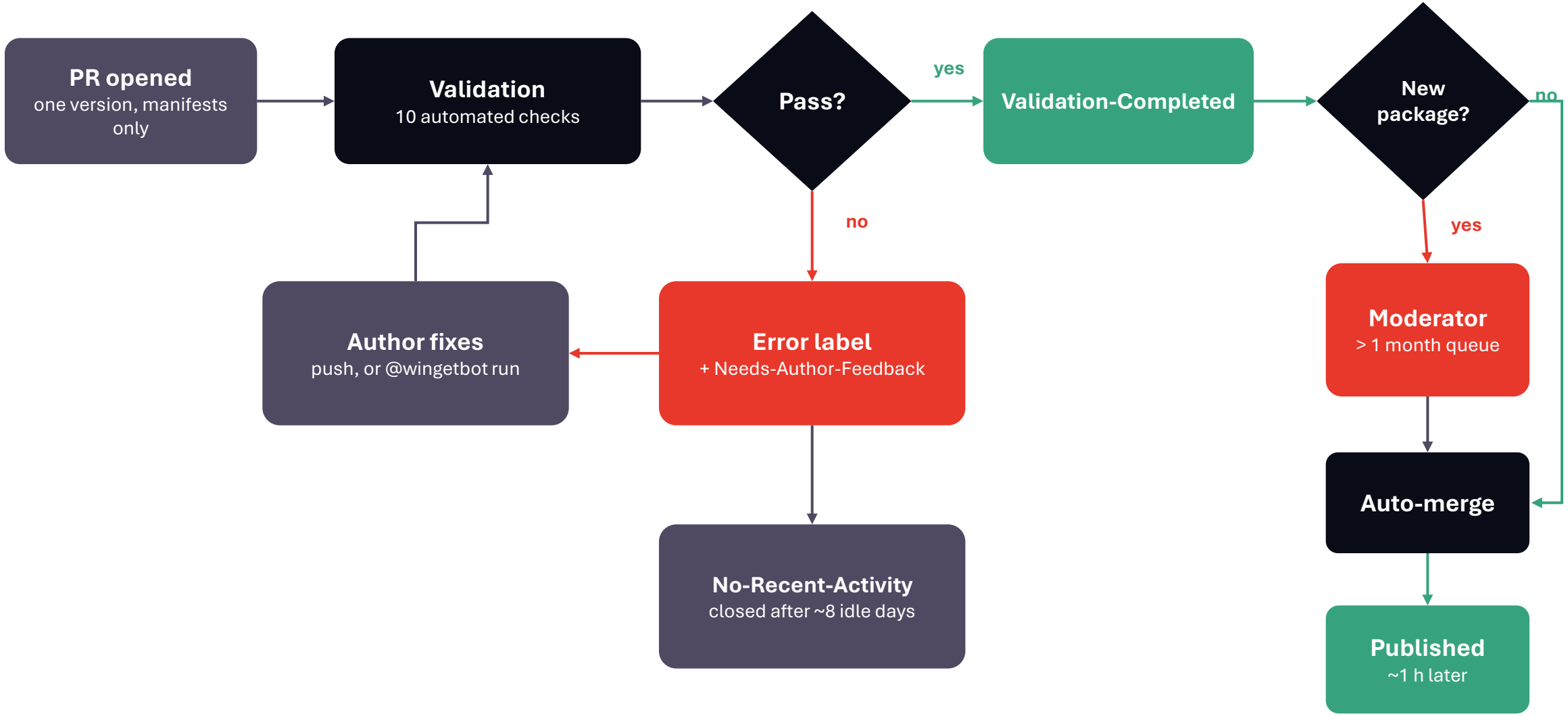


Robots do most of the work

New installer manifests since 1 July 2026, by tool



Now what? The life of a PR



Labels you will meet

Label	Meaning
`Validation-Completed`	every check passed: waiting on a moderator or auto-merge
`Manifest-Validation-Error`	schema, path or rule failure
`URL-Validation-Error / Validation-Domain`	unreachable URL, or not the publisher's official domain
`Binary-Validation-Error`	AV / PUA detection. Cannot be waived
`Validation-Installation-Error`	failed a silent install as a standard (non-admin) user
`Needs-Author-Feedback`	over to you. Idle PRs close after about 8 days
`Moderator-Approved`	new packages only: a human signed it off
`Publish-Pipeline-Succeeded`	in the index, typically within about an hour

What validation proves

Every PR

- Schema + path rules
- URL + publisher domain
- SHA256 matches
- AV + PUA scan
- Silent install, standard user
- Defender scan after install
- App launches after install

New packages only

- Human moderator review
- Screenshot check

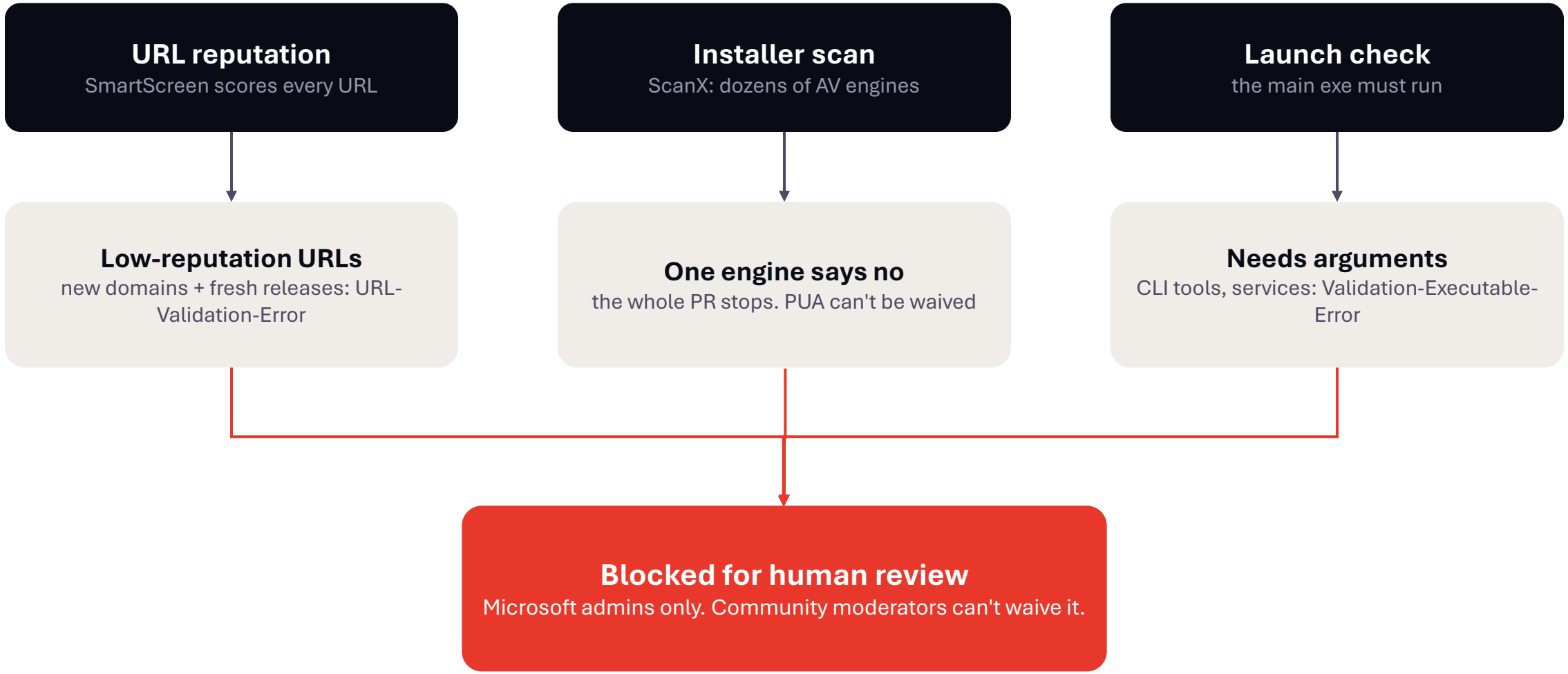
> 1 month

new package review queue, driven by AI agent volume

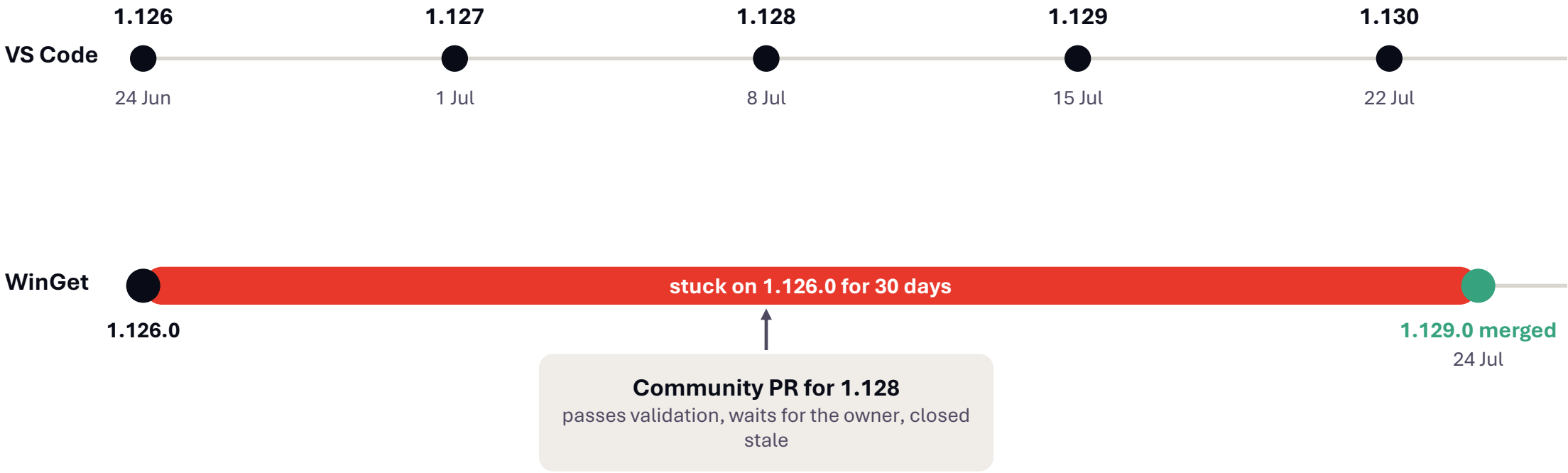
Not checked

- Upgrade over the old version
- Silent uninstall
- Behaviour changes between versions
- Coexistence with your apps
- Metadata accuracy

When the checks go wrong



When the owner goes quiet



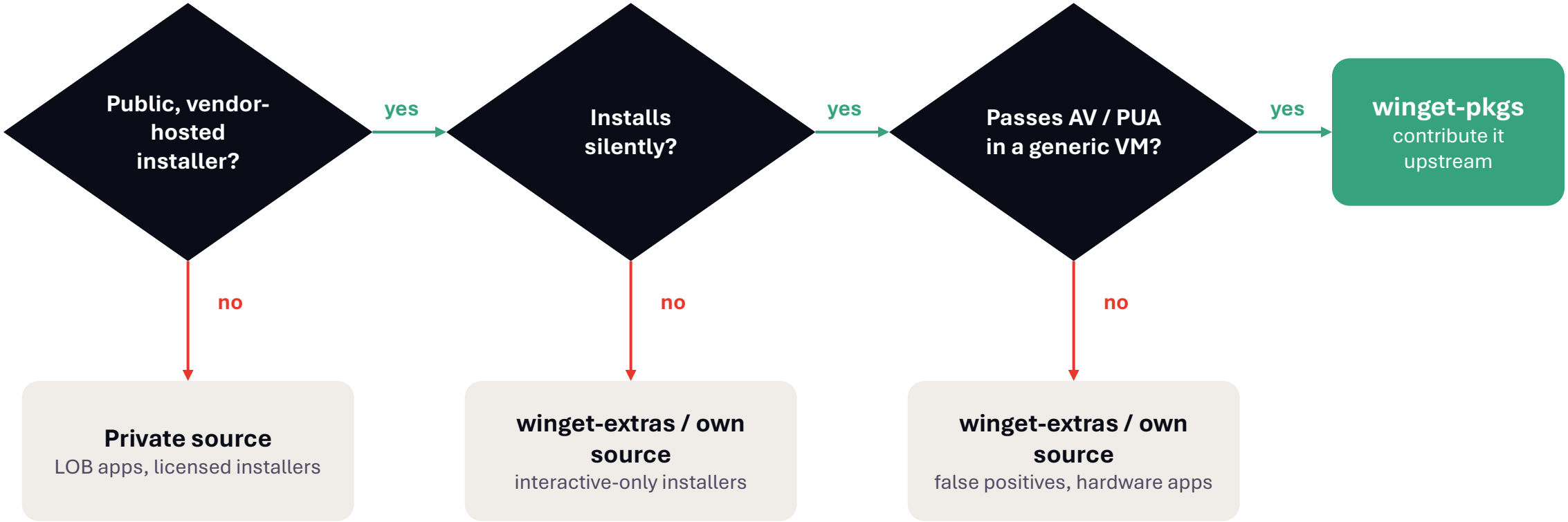
Today: owner approval

A community PR to a maintained package waits for the usual submitter to approve it. If they go quiet, it goes stale.

New: verified developers

An owners file marks the publisher. Their PRs are pre-validated and merge with no moderator. Nobody else can submit.

Where should my app go?



Even in winget-pkgs you don't control timing: pin or gate anything critical.

My app can't pass the checks?

Run your own source. Here is mine.

winget-extras

A public WinGet source for packages that can't go into microsoft/winget-pkgs. Inspired by Scoop Extras.

- Interactive-only installers
- False-positive AV detections
- Hardware-specific apps
- Admin tools like CMTrace and IME

228
apps

181
fonts

1,402
installer manifests

2,062
commits since Jan 2026

Admin PowerShell

```
PS> winget source add --name winget-extras --type Microsoft.PreIndexed.Package `
        --arg https://winget.tplant.com.au/cache
PS> winget search CMTrace -s winget-extras
```

How it works

1 → 2 → 3 → 4 → 5

Shard
JSON / TS: where to find new versions

Anthelion
runs twice an hour, opens PRs via Komac

Validate
real installs on GitHub runners

Auto-merge
bot PRs merge when green

Publish
signed index.db MSIX to Cloudflare R2

Design choices

- No servers: hosting costs about nothing
- Same format + tools as winget-pkgs
- Built with Microsoft's IndexCreationTool

Add a package

- Open a package request issue
- Or run Komac against the repo
- Add a shard to keep it updated

Validation you can see

Matrix — one job per installer: arch x scope x type, arm64 on ARM runners

↓

Snapshot — Attack Surface Analyzer baseline

↓

Install — silent install, 2-minute timeout, screenshot on hang

↓

Diff — everything the installer touched, as SARIF

↓

Launch — start the app, wait, screenshot

Artifacts on every PR

- WinGet + installer logs
- Screenshot of the running app
- SARIF diff

Not tested: 32-bit ARM, hardware-specific apps.

Demo

winget-extras end to end



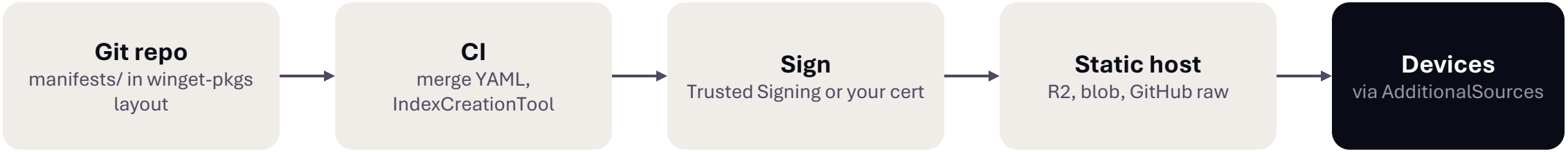
Deploy it to your fleet

```
# Intune Settings Catalog
#   Administrative Templates > Windows Components > Desktop App Installer
#   > Enable App Installer Additional Sources

{
  "Name": "winget-extras",
  "Type": "Microsoft.PreIndexed.Package",
  "Arg": "https://winget.tplant.com.au/cache",
  "Data": "tplant.Winget.Source_ggk937h18f62r",
  "Identifier": "tplant.Winget.Source_ggk937h18f62r",
  "TrustLevel": ["Trusted"],
  "Explicit": false
}

# Same value works in Group Policy. Pair it with EnableAllowedSources
# if you want to lock down what else users can add.
```

Host your own



pl4nty/winget-pkgs-selfhost: a GitHub template that does all of this

Static + public

Any static host. What winget-extras does.

Static + private-ish

Behind a VPN or IP allow-list. The URL is the secret.

REST + Entra ID

winget-cli-restsource on Azure. Per-user auth, not from SYSTEM.

Takeaways

Manifests + installs

Know how manifests drive installs and detection.

Contributing

How to contribute, and where things get held up.

Your own source

Running your own source is pretty easy!

Devicie

Devicie automates Microsoft Intune device management and security, and packages a lot of apps with WinGet along the way.

A Melbourne sponsor today. Come and say hi!

500+ manifests in microsoft/winget-pkgs contributed by us!

DEVICIE

devicie.com

Questions?

Tom Plant | github.com/pl4nty

Thank You, Ninjas!

